Resumo Técnico — E1-TCP01

Servidor concorrente TCP em Java, serialização de objetos e tratamento de falhas

7 de outubro de 2026

Resumo

Este documento resume a arquitetura, o código e a teoria subjacente ao sprint **E1-TCP01**. Cobre as três classes centrais do servidor (`TCPServer`, `Connection`) e do cliente (`TCPClient`), o papel da serialização Java, as quatro falhas deliberadas da etapa 4.4 e as boas práticas a corrigir no código atual.

Conteúdo

1 Arquitetura geral **2**

2 TCPServer.java **2**

3 Connection.java **3**

4 TCPClient.java **5**

5 Serialização **6**

6 As quatro falhas deliberadas (etapa 4.4) **7**

7 Teoria essencial **8**

8 Resumo de correções sugeridas **9**

9 Preparação para a defesa **9**

Conclusão **10**

1 Arquitetura geral

O sprint segue o padrão *one thread per connection*: o servidor tem uma thread principal bloqueada em `accept()`, e para cada cliente aceite cria uma thread dedicada que trata do pedido do início ao fim.

Componente	Responsabilidade
`TCPServer`	Escuta no porto, aceita ligações, cria `Connection`.
`Connection`	Thread que lê o objeto do cliente, processa, responde e fecha o socket.
`TCPClient`	Liga-se, envia uma `Person` (com `Place`), recebe a resposta em texto.
`Person`	Objeto serializável enviado pelo cliente.
`Place`	Objeto serializável, dependência da `Person`.

As classes `Person` e `Place` têm de existir **nos dois projetos**, no mesmo pacote (`tcp01`), com o mesmo `serialVersionUID`. Caso contrário, a desserialização falha.

2 TCPServer.java

```
1  package tcp01;
2
3  import java.io.*;
4  import java.net.*;
5
6  public class TCPServer {
7      public static void main(String[] args) {
8          try {
9              int serverPort = 7896;
10             ServerSocket listenSocket = new ServerSocket(
                   serverPort);
11             while (true) {
12                 Socket clientSocket = listenSocket.accept();
13                 Connection c = new Connection(clientSocket);
14             }
15         } catch (IOException e) {
16             System.out.println("Listen: " + e.getMessage());
17         }
18     }
19 }
```

Listing 1: TCPServer.java

Pontos-chave

- `ServerSocket` é o *posto de escuta*; `Socket` é uma ligação concreta. Com N clientes ligados ao mesmo tempo existem 1 `ServerSocket` e N `Socket`.
- `accept()` é **bloqueante**: enquanto não houver um cliente a ligar-se, o `main` fica parado, sem consumir CPU.
- O `while (true)` é essencial: sem ele, o servidor aceitaria uma única ligação e terminaria.
- Cada cliente aceite é entregue a um novo objeto `Connection`. Como o construtor desta chama `this.start()`, o `main` volta imediatamente ao `accept()`.

Quando é que o `catch (IOException)` é atingido?

O `try` envolve tudo: criação do `ServerSocket`, o `accept()` e o `new Connection(...)`. O `catch` apanha:

- **Criação do `ServerSocket`**: `BindException` (porto ocupado, permissão negada).
- **`accept()`**: `SocketException` (socket de escuta fechado, erro grave de rede).
- **`new Connection(...)`**: raramente, porque o construtor da `Connection` já trata internamente.

Atenção

Um cliente que se desliga abruptamente **não** provoca um erro neste `catch`. O erro (se aparecer) é tratado dentro do `run()` da `Connection` como `EOFException`. Distinguir isto é o ponto central da etapa 4.4.

3 Connection.java

```
1  package tcp01;
2
3  import java.io.*;
4  import java.net.*;
5
6  public class Connection extends Thread {
7      DataInputStream in;
8      DataOutputStream out;
9      Socket clientSocket;
10
11     public Connection(Socket aClientSocket) {
12         try {
13             clientSocket = aClientSocket;
14             in = new DataInputStream(clientSocket.getInputStream
                   ());
15             out = new DataOutputStream(clientSocket.
                   getOutputStream());
16             this.start();
17         } catch (IOException e) {
18             System.out.println("Connection: " + e.getMessage());
19         }
20     }
21
22     @Override
23     public void run() {
24         try {
25             ObjectInputStream ois = new ObjectInputStream(
                   clientSocket.getInputStream());
26             Person p = (Person) ois.readObject();
27             out.writeUTF(p.getPlace().getLocality());
28         } catch (EOFException e) {
29             System.out.println("EOF: " + e.getMessage());
30         } catch (ClassNotFoundException e) {
31             System.out.println("ClassNotFound: " + e.getMessage
                   ());
32         } catch (IOException e) {
33             System.out.println("IO: " + e.getMessage());
```

```
34                  } finally {
35                      try {
36                          clientSocket.close();
37                      } catch (IOException e) {
38                          /* falha ao fechar */
39                      }
40                  }
41              }
42          }
```

Listing 2: Connection.java

Pontos-chave

- `extends Thread`: cada `Connection` é, ela própria, uma thread.
- Os campos `in`, `out` e `clientSocket` são *package-private*. Em projetos maiores deveriam ser `private`.
- O campo `in` está declarado mas **nunca é lido** no `run()` — é um resíduo da etapa 4.1. Poderia ser removido.
- O `run()` cria um `ObjectInputStream` a partir do socket. Isto **substitui** o `DataInputStream` do construtor: só se pode ler de uma stream subjacente de cada vez.
- O `cast (Person)` é obrigatório: `readObject()` devolve `Object` porque, em compilação, o Java não sabe o tipo que vem pelo fio.
- O `Place` não é lido em separado: chega automaticamente dentro da `Person`, porque o Java serializa o grafo de objetos alcançável.

Ordem dos `catch`

`EOFException` é subclasse de `IOException`. A ordem tem de ser da mais específica para a mais genérica:

`EOFException` → `ClassNotFoundException` → `IOException`

Se `IOException` viesse primeiro, o Java daria erro de compilação (*exception already caught*). O `ClassNotFoundException` tem de ser apanhado porque o `run()` é `@Override` de `Thread.run()`, que não declara `throws`.

O `finally`

O `finally` corre sempre, com ou sem exceção. Fecha o socket, libertando o *file descriptor* associado. Sem este fecho, cada cliente deixaria um recurso pendurado até o processo terminar; ao fim de muitos clientes, o servidor esgotaria o limite do sistema operativo e deixaria de aceitar ligações.

Como ficaria correto

Arrancar uma thread dentro do construtor é considerado má prática: o objeto pode não estar totalmente construído quando o `run()` começa a executar, e em classes com subclasses o `run()` sobrescrito pode correr antes de a subclasse se inicializar. O correto é expor um método separado:

```
public Connection(Socket aClientSocket) {
    this.clientSocket = aClientSocket;
    // ... criar streams, sem start()
}

public void startConnection() {
    this.start();
}
```

E no `TCPServer`:

```
Connection c = new Connection(clientSocket);
c.startConnection();
```

4 TCPClient.java

```
package tcp01;

import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        Socket s = null;
        try {
            int serverPort = 7896;
            s = new Socket("localhost", serverPort);

            Place place = new Place("4900-000", "Viana do
                Castelo");
            Person person = new Person("Ana", place, 2003);

            ObjectOutputStream oos = new ObjectOutputStream(s.
                getOutputStream());
            oos.writeObject(person);
            oos.flush();

            DataInputStream in = new DataInputStream(s.
                getInputStream());
            String data = in.readUTF();
            System.out.println("Received: " + data);

        } catch (UnknownHostException e) {
            System.out.println("Sock: " + e.getMessage());
        } catch (EOFException e) {
            System.out.println("EOF: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO: " + e.getMessage());
        } finally {
```

```
31                      if (s != null) {
32                          try { s.close(); }
33                          catch (IOException e) { System.out.println("
                                close: " + e.getMessage()); }
34                      }
35                  }
36              }
37          }
```

Listing 3: TCPClient.java

Pontos-chave

- O `Socket` é declarado **antes** do `try` para que o `finally` lhe possa aceder. Variáveis locais em Java **não** são auto-inicializadas; daí o `= null`.
- O `new Socket("localhost", 7896)` desencadeia o *three-way handshake* do TCP (SYN, SYN-ACK, ACK), feito pelo sistema operativo, sem intervenção do `accept()` do servidor.
- O `ObjectOutputStream` **envia um cabeçalho de stream no próprio construtor** (número mágico, versão do protocolo). Isto é o ponto de sincronização com o `ObjectInputStream` do servidor, que fica bloqueado à espera desse cabeçalho.
- A resposta é lida como texto (`readUTF`), porque o enunciado pede assimetria: objeto num sentido, texto no outro.
- `ConnectException` **não** tem `catch` próprio: é subclasse de `IOException` e cai no `catch` genérico. É por isso que o output mostra `IO: Ligação recusada`.

Como ficaria correto

Para o relatório da 4.4 ser mais claro, podes acrescentar `catch` específicos **antes** dos genéricos:

```
        } catch (ConnectException e) {
            System.out.println("Connect: " + e.getMessage());
        } catch (NotSerializableException e) {
            System.out.println("NotSerializable: " + e.getMessage());
        } catch (InvalidClassException e) {
            System.out.println("InvalidClass: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("IO: " + e.getMessage());
        }
```

A ordem é obrigatória: subclasse antes de superclasse.

5 Serialização

O que é

`Serializable` é uma **interface de marcação** (*marker interface*): não tem métodos nem campos. Serve apenas para autorizar o `ObjectOutputStream` a converter objetos da classe em bytes. Sem esta interface, `writeObject` lança `NotSerializableException`.

O que é serializado

- Campos **não estáticos** e **não `transient`**.
- Todos os objetos referenciados por esses campos (grafo de objetos). É por isso que a `Person` "leva" o `Place` atrás, sem escrita explícita.

`serialVersionUID`

- É um campo `private static final long`, com **nome reservado**. Não é escolha: a JVM procura exatamente esse identificador.
- Deve ser **declarado explicitamente** nos dois lados, com o mesmo valor.
- Se for omitido, o Java calcula-o automaticamente a partir da estrutura da classe. Qualquer alteração (novo campo, novo método, mudança de tipo) muda o UID automático e quebra a desserialização sem aviso.
- UIDs diferentes entre cliente e servidor provocam `InvalidClassException` no `readObject()`.

Identidade da classe

O nome completo (`package + classe`) faz parte dos metadados do objeto serializado. Se a `Person` estiver em `tcp01` no cliente e em `tcp02` no servidor, o `readObject()` lança `ClassNotFoundException`. O nome do ficheiro tem de coincidir com a classe pública; o nome da variável local é irrelevante.

6 As quatro falhas deliberadas (etapa 4.4)

Cada falha deve ser provocada **isoladamente**, com reposição do código correto entre testes.

Alteração	Onde rebenta	Exceção	Instrução
Cliente sem servidor	Cliente	`ConnectException`	`new Socket(...)`
`Place` sem `Serializable`	Cliente	`NotSerializableException`	`oos.writeObject(person)`
`serialVersionUID` diferente	Servidor	`InvalidClassException`	`ois.readObject()`
`Person` ausente no servidor	Servidor	`ClassNotFoundException`	`ois.readObject()`

Notas sobre cada falha

1. **Servidor desligado.** `ConnectException: Connection refused`. O nome `localhost` resolveu bem (não é `UnknownHostException`); simplesmente ninguém estava à escuta no porto 7896. O output do cliente será `IO: Ligação recusada`, porque `ConnectException` é apanhada pelo `catch (IOException)` genérico.
2. **`Place sem Serializable`.** O `ObjectOutputStream` percorre o grafo da `Person`, encontra o campo `place` e verifica que a classe não é serializável. Atira `NotSerializableException` **antes** de enviar seja o que for pela rede. O servidor nem chega a receber nada.
3. **UID diferente.** O cliente serializa com UID 1, o servidor tem UID 2. O `ObjectInputStream` compara a assinatura do fluxo com a da classe local, não coincidem, e o Java recusa a desserialização com `InvalidClassException`.

4. **`Person` ausente no servidor.** Ao ler o fluxo, o `ObjectInputStream` encontra a referência `tcp01.Person` e procura-a no classpath. Como não existe (ou existe noutro pacote), lança `ClassNotFoundException`.

Atenção

Distinção importante: `EOFException` no servidor **não é erro**. Significa que o cliente fechou a ligação sem enviar o objeto completo, e é o comportamento normal quando o cliente termina. Deve ser distinguida de erros de rede ou de serialização.

7 Teoria essencial

O que o TCP garante

- **Entrega fiável**: os bytes chegam todos (retransmissão em caso de perda).
- **Ordem**: os bytes chegam pela ordem em que foram enviados.
- **Deteção de erros**: *checksums* por segmento.
- **Controlo de fluxo e congestionamento**.

O que o TCP não garante

- **Fronteiras de mensagem**: o TCP vê um fluxo contínuo de bytes (*byte stream*). Cabe à aplicação definir onde termina uma mensagem (`readUTF`, `readObject`).
- **Significado**: os bytes não têm semântica para o TCP. Só a aplicação sabe se representam um `int`, uma `String` ou uma `Person`.

Daí a separação de responsabilidades:

TCP transporta bytes com fiabilidade e ordem $\neq$ **Aplicação** interpreta-os.

Threads no servidor

Com `N` clientes ligados ao mesmo tempo:

- 1 thread `main` (bloqueada em `accept()`).
- `N` threads `Connection` (uma por cliente).
- Total: `N+1` threads vivas.

Se trocarmos `this.start()` por `this.run()` no construtor, o `run()` corre na thread `main`, e o servidor passa a atender clientes em série. Os clientes seguintes ficam pendentes na fila do `ServerSocket` (o *backlog*) até o `main` voltar ao `accept()`.

Limitações identificadas (CA5)

- Uma thread por cliente não escala para milhares de clientes simultâneos: cada thread consome memória e tempo de escalonamento.
- `transient` não é abordado nesta ficha: sem ele, não há forma de impedir que um atributo sensível (ex.: password) seja serializado e viaje pela rede.
- O servidor não trata de falhas no `accept()` de forma resiliente: se o `accept()` falhar, o `while` termina e o servidor morre.

8 Resumo de correções sugeridas

Local	Problema	Correção
`Connection`	Campos sem `private`	Acrescentar `private` a `in`, `out`, `clientSocket`
`Connection`	Campo `in` declarado mas nunca lido	Remover do construtor e da classe
`Connection`	`this.start()` dentro do construtor	Mover para método `startConnection()`, chamado pelo `TCPServer`
`Connection`	`ClassCastException` não apanhada	Acrescentar `catch` específico se for relevante para o teste
`TCPClient`	Falta `catch` específico para `ConnectException`	Acrescentar para o relatório da 4.4
`TCPClient`	`ConnectException` confundida com `IOException`	Documentar no relatório

9 Preparação para a defesa

Perguntas prováveis e respostas

1. **Porque é que a `Connection` estende `Thread` e o que aconteceria se chamasses `run()` em vez de `start()`?**
   `start()` cria uma thread nova e devolve o controlo ao `main`, que volta ao `accept()`. `run()` executaria na thread `main`, bloqueando-a até o cliente terminar. Com dois clientes, o segundo ficaria pendente na fila do `ServerSocket`.

2. **Porque é que o cast `(Person)` é obrigatório?**
   `readObject()` devolve `Object`, porque em compilação o Java não sabe o tipo que vem pelo fio. O cast diz ao compilador para tratar como `Person`; se o objeto recebido não for uma `Person`, dá `ClassCastException` em runtime.

3. **Porque é que o `Place` chega ao servidor sem ser escrito em separado?**
   O Java serializa o grafo de objetos alcançável. Ao escrever a `Person`, o `ObjectOutputStream` deteta que o campo `place` é `Serializable` e inclui-o automaticamente.

4. **O que acontece se o `serialVersionUID` for diferente nos dois lados?**
   `InvalidClassException` no `readObject()`, porque a assinatura do fluxo não corresponde à da classe local.

5. **O que acontece se a `Person` estiver num pacote diferente no servidor?**
   `ClassNotFoundException` no `readObject()`, porque o nome completo (`package + classe`) faz parte dos metadados e não é encontrado no classpath.

6. **Quantos sockets e threads existem no servidor com dois clientes ligados em simultâneo?**
   Sockets: 1 `ServerSocket` + 2 `Socket`. Threads: 1 `main` + 2 `Connection` = 3.

7. **O `ObjectOutputStream` envia alguma coisa no construtor?**
   Sim, um cabeçalho de stream (número mágico, versão do protocolo). Isto é o ponto de sincronização com o `ObjectInputStream` do servidor, que fica bloqueado à espera desse cabeçalho.

8. **Porque é que `EOFException` no servidor não é um erro?**
   Significa que o cliente fechou a ligação sem enviar o objeto completo. É o comportamento normal quando o cliente termina e já não envia mais dados. Distingue-se de erros de rede ou de serialização.

9. **Qual é a limitação principal deste modelo para 10 000 clientes?**
   Uma thread por cliente não escala: cada thread consome memória (pilha) e tempo de escalonamento. Modelos mais escaláveis usam NIO (*selector*) ou um *thread pool* com um número fixo de threads.

10. **Porque é que o `Place` tem de implementar `Serializable` e não apenas a `Person`?**
    A serialização percorre o grafo completo. Basta um objeto do grafo não ser `Serializable` para que todo o `writeObject` falhe com `NotSerializableException`.

Onde estão os erros de cada cliente?

- Cliente não consegue ligar-se: `ConnectException` ou `UnknownHostException` **no cliente**.
- Erro de serialização: `NotSerializableException` **no cliente**, ao fazer `writeObject`.
- Erro de desserialização: `InvalidClassException` ou `ClassNotFoundException` **no servidor**, ao fazer `readObject`.
- Cliente desliga-se abruptamente: `EOFException` **no servidor**, dentro do `run()`.
- Porto ocupado: `BindException` **no servidor**, no `new ServerSocket(...)`.

Conclusão

O sprint **E1-TCP01** implementa o padrão *one thread per connection* em Java, com três classes principais no servidor e cliente, e duas classes de domínio (`Person` e `Place`) que atravessam a rede por serialização Java. As três etapas funcionais (4.1 texto, 4.2 objeto simples, 4.3 objeto com dependência) estão concluídas e validadas. A etapa 4.4 é servida por quatro falhas deliberadas que expõem os pontos de rutura do protocolo de serialização. A etapa 4.5 (reflexão) encontra neste documento matéria suficiente para as respostas.

As duas principais correções a considerar antes da defesa são:

1. retirar `this.start()` do construtor da `Connection` e expor um método `startConnection()`;
2. limpar o campo `in` da `Connection`, que ficou como resíduo da etapa 4.1.